01 / Base común

VetRed · Guía breve para defender el TP en preguntas y respuestas

Objetivo: los tres deben entender el sistema completo. El reparto indica quién profundiza y quién responde primero; no divide el conocimiento en partes aisladas.

Qué pide el enunciado

Integrar clientes, mascotas e historias clínicas, atenciones y derivaciones, empaquetado, transporte, recepción, resultados y comprobantes. Exige las cinco validaciones de negocio, al menos **dos patrones** y **cuatro reportes no triviales**. Evalúa especialmente el ciclo de vida de muestras y remitos.

Explicación común — 20 segundos

“VetRed conecta la atención veterinaria con el laboratorio. Una consulta puede generar una orden y muestras, que se empaquetan, transportan y reciben con controles sanitarios. Registramos resultados, trazabilidad y comprobantes, y generamos reportes de gestión.”

Mapa que todos deben conocer

Interfaz → Servicios → DAO → SQLite. El dominio contiene entidades y comportamiento; los servicios coordinan operaciones. La interfaz no ejecuta SQL.

Circuito: cliente y mascota → atención → orden y extracción → remito y precinto → envío → recepción → resultados y comprobante. La facturación no necesita esperar a que termine el análisis.

POO y patrones, en una frase

• **Herencia y abstracción:** Persona es abstracta; Cliente, Veterinario y Bioquimico especializan su rol.

• **Encapsulamiento y polimorfismo:** las muestras delegan operaciones en estados; las tarifas comparten una interfaz y calculan distinto.

• **State:** acciones según estado. **Strategy:** cálculo de tarifa. **Iterator:** recorrido de historiales. **Singleton:** conexión única. **DAO:** acceso a datos separado.

Cómo responder y practicar

Qué hace → por qué → archivo y método → ejemplo o prueba. Primero responde el responsable; otro complementa. Practiquen cinco preguntas cruzadas por persona y dos casos: recepción normal y precinto roto.

Referencia de esta guía: main, commit 86c0906, revisado el 12/11/2026. Todas las rutas parten de veterinaria/. Antes de defender, acuerden una versión y revisen las diferencias con otras ramas. El enunciado del tema no fija formato ni duración de la defensa.

02 / Agustín

Especialidad: atención clínica y relaciones entre objetos · Respaldo: Francisco

Qué debe poder explicar

• Cómo se vinculan cliente, mascota, historia clínica, atención y orden de derivación.

• Qué es herencia, qué aporta una clase abstracta y dónde se ve polimorfismo en Persona.rol().

• Dónde se valida el responsable, la agenda y el esquema de vacunas; qué ocurre si una operación falla.

Respuesta de apertura

“Mi parte cubre la atención clínica. Toda mascota tiene un responsable y al registrarla abrimos su historia clínica. Las atenciones reúnen tratamientos, vacunas y órdenes. Las reglas se validan en dominio y servicios; una transacción permite guardar operaciones relacionadas como una unidad.”

Código para tener ubicado

app/dominio/personas.py
Persona (ABC), Cliente, Veterinario y Bioquimico; métodos rol() y validar().

app/servicios/mascota_service.py
MascotaService.alta(): valida responsable y guarda mascota e historia en una transacción.

app/dominio/turnos.py + app/servicios/turno_service.py
Turno.se_superpone_con(), Agenda.reservar() y TurnoService.reservar().

app/servicios/atencion_service.py
AtencionService.registrar(): vacunas, turno, atención y derivación opcional.

app/presentacion/vistas/atenciones_vista.py
Punto de entrada desde la pantalla; seguir la llamada al servicio.

Preguntas probables y respuesta breve

¿Por qué Persona es abstracta?
Agrupa datos y comportamiento comunes; exige que cada clase concreta defina rol().

¿Cómo evitan turnos superpuestos?
Comparan intervalos en la agenda y refuerzan la regla con triggers de SQLite. Cancelar un turno libera su horario.

¿Qué ocurre si faltan vacunas?
Se detectan las pendientes y se pide confirmación para continuar. No es un bloqueo absoluto de la atención.

¿Por qué usar una transacción?
Para que, si falla una operación, no quede guardada solo una parte del registro relacionado.

Qué mostrar y qué tener en cuenta

Mostrar alta de mascota y atención con vacunas pendientes. Distinguir **herencia** (Cliente es Persona) de **asociación** (Cliente tiene mascotas). No presentar todas las relaciones como herencia.

tests/test_servicios.py
test_alta_de_mascota_abre_historia_clinica · test_turnos_sin_superposicion · test_control_de_esquema_de_vacunas

03 / Francisco

Especialidad: trazabilidad, State e Iterator · Respaldo: Lucas

Qué debe poder explicar

• Cómo una orden genera muestras y cómo se agrupan en una conservadora y un remito con precinto.

• Por qué muestra y remito tienen ciclos distintos y cómo el remito coordina acciones sobre sus muestras.

• Cómo se rechazan muestras y cómo se conserva y recorre su historial.

Respuesta de apertura

“El núcleo del trabajo es la trazabilidad. Usamos State para que cada estado determine qué operaciones admite una muestra o un remito. En recepción verificamos precinto, temperatura y viabilidad. Cada cambio conserva fecha, responsable y motivo; el historial se recorre con Iterator y la base impide modificarlo o borrarlo.”

Código para tener ubicado

app/dominio/muestra.py + app/dominio/estados_muestra.py
Muestra es el contexto. EstadoMuestra define las operaciones; Extraida, Empaquetada, EnTransito, Recibida, Analizada y Rechazada las especializan.

app/dominio/remito.py + app/dominio/estados_remito.py
RemitoBiologico y sus estados. EnTransito.recibir() coordina aceptación o rechazo de muestras.

app/dominio/historial.py
CambioEstado es frozen; HistorialEstados agrega cambios e IteradorHistorial implementa __iter__() y __next__().

app/servicios/derivacion_service.py + app/servicios/laboratorio_service.py
Precintar y despachar; LaboratorioService.registrar_recepcion() y registrar_resultado().

app/persistencia/schema.sql
Triggers que impiden UPDATE y DELETE del historial de estados de muestras.

Preguntas probables y respuesta breve

¿Por qué State y no solo un campo de texto?
El estado también define comportamiento. Las acciones inválidas generan TransicionInvalidaError.

¿Qué ocurre si llega el precinto roto?
Se rechaza el remito y todas sus muestras; no se permite procesarlas.

¿Un remito recibido puede tener muestras rechazadas?
Sí. Con precinto intacto se evalúa temperatura y viabilidad por muestra; algunas pueden rechazarse.

¿Para qué Iterator y cómo protegen el historial?
Iterator permite recorrerlo sin exponer la lista interna. La protección combina registros inmutables, operaciones de agregado y triggers en la base.

Qué mostrar y qué tener en cuenta

Recorrer **Extraída → Empaquetada → En tránsito → Recibida → Analizada**. Mostrar luego precinto roto. Una nueva extracción crea otra muestra: no reactiva la rechazada. El prefijo “_” en Python es una convención, no privacidad absoluta.

tests/test_estado_muestra_remito.py + tests/test_servicios.py
test_transicion_invalida · test_precinto_roto_rechaza_todas_las_muestras · test_iterador_explicito · test_trazabilidad_inmutable_en_la_base

04 / Lucas

Especialidad: arquitectura, persistencia y facturación · Respaldo: Agustín

Qué debe poder explicar

- Cómo se separan interfaz, servicios, dominio y persistencia, y dónde se conectan sus dependencias.
- Cómo Strategy cambia el cálculo del comprobante; qué hacen Singleton y DAO.
- Qué información aportan los reportes y qué verifican las pruebas automáticas.

Respuesta de apertura

“Separamos responsabilidades para que la interfaz no dependa de consultas SQL. Los servicios coordinan reglas y los DAO persisten objetos en SQLite. En facturación usamos Strategy para elegir la tarifa de consulta, control o urgencia. Los reportes cruzan datos clínicos, logísticos y económicos; las pruebas verifican las reglas principales.”

Código para tener ubicado

app/servicios/contenedor.py
Servicios crea los DAO y servicios y conecta sus dependencias.

app/dominio/tarifas.py + app/dominio/facturacion.py
EstrategiaTarifa, TarifaConsulta, TarifaControl y TarifaUrgencia; ComprobanteAtencion delega el cálculo.

app/servicios/facturacion_service.py
estrategia_para() elige mediante un diccionario de lambdas; preparar() y emitir() construyen y guardan el comprobante.

app/persistencia/conexion.py + app/persistencia/daos/base_dao.py
ConexionBD usa __new__() para Singleton. transaccion() coordina commit/rollback. BaseDAO reúne operaciones comunes y delega el mapeo.

app/servicios/reporte_service.py + app/reportes/exportador.py
Generación de indicadores y secciones; exportación PDF y CSV.

Preguntas probables y respuesta breve

¿State y Strategy son lo mismo?
State cambia el comportamiento según el ciclo de vida. Strategy selecciona un algoritmo intercambiable, como el cálculo de una tarifa.

¿Cómo varían las tarifas?
Consulta usa precio de lista; control descuenta servicios clínicos; urgencia recarga esos servicios y contempla horario nocturno.

¿Qué separa un DAO de un servicio?
El DAO consulta y guarda datos. El servicio coordina el caso de uso, las reglas y las operaciones necesarias.

¿Qué reportes tienen?
Atenciones y derivaciones; rechazos; tratamientos y vacunas pendientes; facturación; tiempos de tránsito y procesamiento. Incluyen cálculos y agrupaciones.

Qué mostrar y qué tener en cuenta

Comparar estrategias y abrir un reporte con datos. Explicar por qué Singleton centraliza la conexión y cómo un lock protege su uso. La base es local: no hay autenticación ni sincronización entre clínicas.

Verificación: main aprobó 33 pruebas de dominio y servicios. Se ejecutan con **python -m pytest** desde veterinaria/. Esto no equivale a comprobar todas las pantallas o exportaciones. BaseDAO también muestra Método Plantilla: reutiliza un flujo común y especializa el mapeo.